Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

LDP: An Identity-Aware Protocol for Multi-Agent LLM Systems

Sunil Prakash

sunil_prakash_pgpmax2026@isb.edu

Indian School of Business

Research Article

Keywords: agent protocols, multi-agent systems, LLM delegation, payload negotiation, AI interoperability, trust domains

Posted Date: March 19th, 2026

DOI: https://doi.org/10.21203/rs.3.rs-9121599/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

LDP: An Identity-Aware Protocol for Multi-Agent LLM Systems

Sunil Prakash[1*]

[1*]Indian School of Business, Hyderabad, India.

Corresponding author(s). E-mail(s):
sunil_prakash_pgpmax2026@isb.edu;

Abstract

As multi-agent AI systems scale to production, the protocols connecting them constrain their capabilities. Current protocols such as Google's A2A and Anthropic's MCP omit model-level metadata, preventing quality-aware routing, efficient communication, and governance. We present the LLM Delegate Protocol (LDP), which introduces five AI-native primitives: rich delegate identity cards, progressive payload negotiation, governed sessions, structured provenance, and trust domains. We implement LDP as a plugin adapter for the JamJet agent runtime and evaluate it against A2A and random baselines. Identity-aware routing achieves 12x lower latency on easy tasks; semantic frame payloads reduce token count by 37 percent with no quality loss; governed sessions eliminate 39 percent token overhead; and noisy provenance degrades quality below the no-provenance baseline, showing that confidence metadata is harmful without verification. Trust domain and fallback analyses demonstrate architectural advantages in security and resilience. This paper contributes a protocol design, reference implementation, and initial evidence that AI-native protocol primitives enable more efficient and governable multi-agent delegation.

Keywords: agent protocols, multi-agent systems, LLM delegation, payload negotiation, AI interoperability, trust domains

1 Introduction

The rapid deployment of large language models (LLMs) in production systems has given rise to multi-agent architectures where specialized AI agents collaborate to solve complex tasks [10, 12, 16]. These systems require communication protocols that

enable agents to discover one another, negotiate interaction terms, delegate tasks, and synthesize results.

Two dominant paradigms have emerged. Google's Agent-to-Agent Protocol (A2A) [8] provides a service-oriented interface where agents expose "Agent Cards" containing a name, description, and list of skills. Anthropic's Model Context Protocol (MCP) [1] focuses on tool-level integration, allowing hosts to invoke functions on tool servers. Both protocols deliberately omit model-level metadata—a design choice favoring simplicity and generality.

However, this opacity discards information critical to effective delegation. When a router must choose between a 3-billion-parameter model optimized for classification and an 8-billion-parameter model with deep reasoning capabilities, knowing only their skill names ("classification", "reasoning") is insufficient. The router cannot assess quality–cost tradeoffs, negotiate communication formats, verify provenance of outputs, or maintain governed multi-round contexts.

We argue that agent-to-agent protocols that are *AI-native*—that expose model identity, negotiate communication richness, enforce governance, and track provenance—can enable more efficient and governable delegation than protocols that do not expose these properties. To explore this thesis, we present the **LLM Delegate Protocol** (LDP), a protocol design with reference implementation and initial empirical study. LDP is designed from first principles around three observations:

1. **AI delegates have rich, actionable identity.** Model family, parameter count, reasoning profile, context window, and cost characteristics are all properties that inform delegation decisions. Exposing them in a structured identity card enables quality-aware routing.
2. **Communication overhead is a first-order cost.** Token consumption directly determines latency and monetary cost. Negotiating compact payload formats (structured semantic frames vs. verbose natural language) yields significant efficiency gains.
3. **Governance requires protocol support.** Multi-round sessions, provenance tracking, trust boundaries, and policy enforcement cannot be reliably retrofitted onto stateless, opaque protocols.

LDP is implemented as an external plugin for the JamJet agent runtime [11], registering alongside existing A2A and MCP adapters with zero modifications to the host system. We evaluate LDP against A2A and random-routing baselines across six research questions, using local Ollama [14] models for delegates and Google Gemini [9] as an LLM judge. Our evaluation combines empirical experiments (routing, payload, provenance, sessions) with simulated protocol analyses (security, fallback).

Contributions.

- The LDP protocol specification: AI-native identity cards, progressive payload modes with negotiation and fallback, governed sessions, structured provenance, and trust domains (§3).
- A reference implementation as a JamJet protocol adapter (§4).

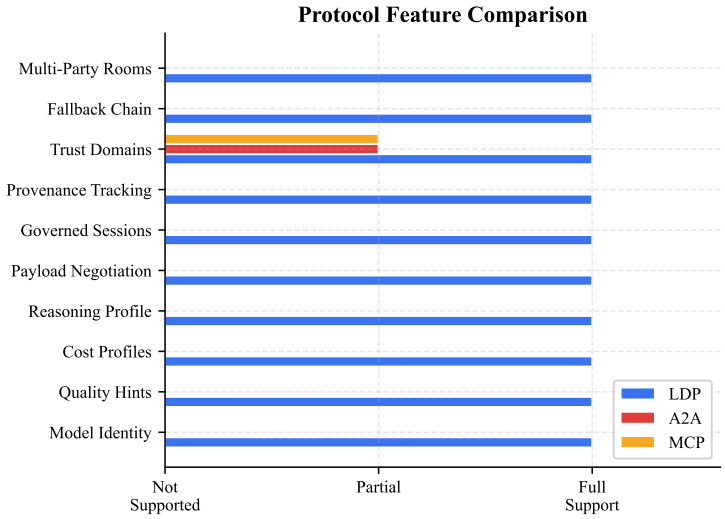


Fig. 1 Feature comparison across agent communication protocols. LDP provides full support for all AI-native features, while A2A and MCP offer only partial transport-level security.

- An initial empirical study with ablation analysis showing that identity metadata and structured payloads improve routing quality and communication efficiency (§6), plus simulated protocol analyses of security and fallback properties.

2 Background and Related Work

2.1 Agent Communication Protocols

Agent communication has a long history in multi-agent systems. The Knowledge Query and Manipulation Language (KQML) [6] and FIPA Agent Communication Language (FIPA-ACL) [7] established foundations for performative-based messaging between software agents in the 1990s and 2000s. These protocols defined speech acts (inform, request, propose) but predated the era of large language models and did not address model-specific properties.

Modern protocols for LLM-based agents include:

Agent-to-Agent Protocol (A2A).

Google's A2A [8] defines a service-oriented protocol for agent interoperability. Agents publish Agent Cards containing a name, description, version, URL, and list of skills. Communication is stateless: clients send tasks and receive results. A2A provides a clean, general interface but deliberately omits model-level metadata, quality signals, session support, provenance, and trust domain enforcement.

Model Context Protocol (MCP).

Anthropic's MCP [1] provides a standardized interface for connecting AI assistants to external tools and data sources. MCP operates at the tool level—hosts invoke functions on servers—rather than the agent level. It is complementary to A2A and LDP but does not address agent-to-agent delegation.

Agent Network Protocol (ANP).

ANP [2] focuses on discovery and networking between agents, providing a directory layer but minimal communication semantics.

LDP builds on the A2A pattern (service discovery, task submission, result retrieval) while extending it with AI-native awareness (§3).

2.2 Multi-Agent Frameworks

Several frameworks orchestrate multi-agent collaboration. AutoGen [16] provides a conversation-based framework with flexible agent topologies. MetaGPT [10] assigns software engineering roles to LLM agents. CAMEL [12] investigates role-playing for cooperative agent behavior. CrewAI [4] emphasizes role-based agent teams with task delegation.

It is important to distinguish three layers in multi-agent systems: *protocols* (how agents communicate), *frameworks* (how agents are orchestrated), and *routers* (which agent to select). These frameworks define *orchestration patterns* but rely on ad-hoc communication (typically function calls or string passing) for the protocol layer. LDP provides the *communication layer* that such frameworks lack—any of them could use LDP as their inter-agent protocol to gain identity-aware routing, payload negotiation, and governance. A natural question is whether A2A could be extended with custom metadata fields rather than adopting a new protocol; we address this directly in §7.4.

2.3 LLM Routing and Delegation

Routing queries to appropriate models is an active research area. FrugalGPT [3] proposes cascading LLM calls to reduce cost while maintaining quality. RouterLLM [15] trains a lightweight router to direct queries to strong or weak models based on difficulty. These approaches optimize *which model to call* but do not address *how to communicate* with it once selected. LDP's identity model provides the metadata that routers need (quality hints, cost profiles, capability manifests) as a protocol primitive rather than a separate system.

2.4 Structured Communication for LLM Agents

Du *et al.* [5] show that multi-agent debate improves factuality. Liang *et al.* [13] find that diverse agent roles improve reasoning quality. These works demonstrate that *how* agents communicate affects output quality—supporting LDP's thesis that richer protocol semantics (structured payloads, identity context) yield better outcomes.

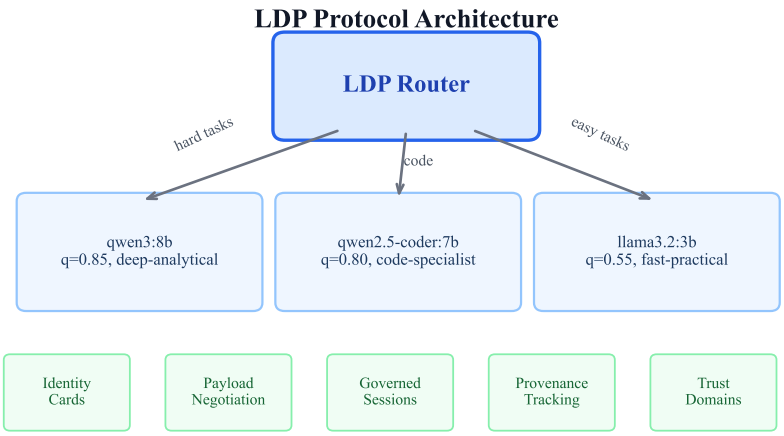


Fig. 2 Architecture of the LLM Delegate Protocol. The LDP router uses rich identity cards (model family, quality hints, reasoning profiles) to make intelligent routing decisions—sending easy tasks to lightweight models and hard tasks to capable models. Five protocol mechanisms work together: identity cards, payload negotiation, governed sessions, provenance tracking, and trust domains.

3 The LLM Delegate Protocol

LDP is designed around a central premise: *agent communication protocols should be aware that their participants are AI models with measurable, heterogeneous properties.* We describe the key protocol mechanisms below.

3.1 AI-Native Identity Model

A2A's Agent Card exposes seven fields: name, description, version, URL, skills, authentication schemes, and supported protocols. LDP extends this with a **Delegate Identity Card** containing 20+ fields organized into four categories:

Core Identity.

`delegate_id`, `principal_id`, `model_family`, `model_name`, `model_version`, `runtime_version`, `weights_fingerprint`, `endpoint_address`.

Trust & Security.

`trust_domain`, `public_key`, `jurisdiction`, `data_handling_policy`.

Capabilities.

`context_window`, `modalities_supported`, `languages_supported`, `tokenizer_fingerprint`. Each capability entry carries `quality_hint` (0–1 continuous score), `latency_hint_ms_p50`, and `cost_hint` metadata.

Behavioral.

`reasoning_profile` (a qualitative descriptor such as "deep-analytical" or "fast-practical") and `cost_profile` ("low", "medium", "high").

This rich identity enables *metadata-aware routing*: a router can send easy classification tasks to a fast, cheap 3B-parameter model and reserve hard reasoning tasks for a slower, more capable 8B model—decisions impossible with A2A's skill-name-only matching.

3.2 Progressive Payload Modes

Token count is a first-order cost in LLM systems. LDP defines six payload modes of increasing efficiency:

- **Mode 0 — Text.** Natural language. Universal compatibility, easiest to audit, highest token count.
- **Mode 1 — Semantic Frames.** Typed structured JSON with explicit fields (`task_type`, `instruction`, `expected_output_format`). Reduces verbosity while remaining human-readable.
- **Mode 2 — Embedding Hints.** Vector representations attached as retrieval or routing signals.
- **Mode 3 — Semantic Graphs.** Structured relationship representations for planning and formal reasoning.
- **Mode 4 — Latent Capsules.** Compressed machine-native semantic packets between compatible models.
- **Mode 5 — Cache Slices.** Execution-state transfer between closely compatible model instances.

Modes 2–3 are fully specified but not yet evaluated empirically. Modes 4–5 require model-level access not universally available through current APIs. We position Modes 2–5 as future work and focus evaluation on Modes 0–1, which are sufficient to demonstrate the payload efficiency thesis.

Negotiation and Fallback.

During session establishment, delegates negotiate the richest mutually supported mode. If a higher mode fails mid-exchange (*e.g.*, schema validation error), the protocol automatically falls back: Mode $N \to$ Mode $N{-}1 \to \cdots \to$ Mode 0. Every delegate **must** support at least Mode 0 (text), ensuring communication never fails entirely.

3.3 Governed Sessions

A2A is stateless: each task submission is independent. For multi-round delegation (iterative refinement, progressive analysis, verification chains), context must be retransmitted with every request, incurring quadratic token overhead.

LDP introduces **governed sessions**—negotiated, persistent contexts with explicit lifecycle:

1. `HELLO`: Caller announces identity.
2. `CAPABILITY_MANIFEST`: Callee responds with supported modes and constraints.
3. `SESSION_PROPOSE`: Caller proposes session parameters (payload mode, latency target, cost budget, privacy constraints, audit level).
4. `SESSION_ACCEPT`: Callee confirms, possibly with adjusted parameters.

5. `TASK_SUBMIT` / `TASK_UPDATE` / `TASK_RESULT`: Task exchange within the session context.

Sessions maintain server-side context, eliminating the need to re-send conversation history. Budget tracking, privacy scoping, and quality-level negotiation persist across rounds.

3.4 Structured Provenance

Every LDP task result carries structured provenance metadata:

```
{
  "produced_by": "delegate:qwen3-8b",
  "model_version": "qwen3-8b-2026.01",
  "payload_mode_used": "semantic_frame",
  "confidence": {"score": 0.84, "method": "self-report"},
  "verification": {"performed": true, "status": "passed"}
}
```

This enables downstream consumers to weight outputs by source reliability—a synthesizer combining opinions from three delegates can prioritize the one with higher confidence and verified status. A2A provides no provenance beyond task completion status.

3.5 Trust Domains

LDP defines trust domains as security boundaries within which identity, policy, and transport guarantees are recognized. Security is enforced at multiple levels:

1. **Message level:** Per-message signatures, nonces, and replay protection.
2. **Session level:** Trust domain compatibility checks during session establishment.
3. **Policy level:** Policy engine validates each task against configurable rules (capability scope, jurisdiction compliance, cost limits).

A2A relies solely on transport-level authentication (bearer tokens), which cannot distinguish trust domains, detect capability escalation, or enforce per-task policies.

3.6 Multi-Party Room Model

Beyond point-to-point delegation, LDP specifies a **room model** with formal roles: Chair (coordination), Recorder (state management), Verifier (output checking), Arbitrator (conflict resolution), and Participants (specialist delegates). This enables structured group collaboration patterns that A2A's point-to-point topology cannot express. The room model is part of the protocol specification but is *not evaluated* in this paper; empirical validation of multi-party coordination is deferred to future work.

4 Implementation

We implement LDP as an external plugin for the JamJet agent runtime [11], a production Rust-based system that orchestrates AI agent workflows. JamJet provides

a `ProtocolAdapter` trait with methods for discovery, invocation, streaming, status checking, and cancellation. Existing adapters implement MCP and A2A.

4.1 Plugin Architecture

LDP registers at runtime via a single function call:

```
let mut registry = default_protocol_registry();
register_ldp(&mut registry, config);
```

URL-based dispatch routes `ldp://` prefixed addresses to the LDP adapter. No modifications to JamJet's core are required—LDP is a pure external dependency.

4.2 Identity Card Extension

JamJet's `AgentCard` structure includes a `labels: HashMap<String, String>` field. LDP identity fields are stored as `ldp.*` labels (*e.g.*, `ldp.model_family`, `ldp.reasoning_profile`, `ldp.cost_profile`), making them accessible to JamJet's existing routing and scheduling infrastructure without schema changes.

4.3 Session Management

Session lifecycle (negotiation, caching, teardown) is encapsulated within the adapter. From JamJet's perspective, each `invoke()` call is stateless. Internally, the adapter maintains a session cache keyed by `(url, session_config)`, creating new sessions on demand and reusing existing ones for multi-round exchanges.

4.4 Model Adapters

To support the experimental evaluation, we implemented two new model adapters for JamJet's `ModelAdapter` trait: an **Ollama adapter** for local inference (supporting Qwen, Llama, Gemma, and Phi model families) and a **Google Gemini adapter** for cloud-based judge evaluation. Both implement structured output support, OpenTelemetry tracing, and prefix-based model routing (*e.g.*, `qwen*` → Ollama, `gemini-*` → Google).

5 Experimental Setup

We evaluate LDP against baselines across six research questions. All experiments use local Ollama models for delegate inference (zero API cost) and Google Gemini 2.5 Flash as the LLM judge.

5.1 Research Questions

RQ1 — *Routing Quality.* Does AI-native identity improve delegation routing quality compared to skill-matching and random selection?
RQ2 — *Payload Efficiency.* Do semantic frame payloads reduce communication cost while preserving quality?

RQ3 — *Provenance Value.* Does structured provenance improve downstream decision quality in multi-source synthesis?
RQ4 — *Session Efficiency.* Do governed sessions reduce token overhead compared to stateless re-invocation in multi-round delegation?
RQ5 — *Security Boundaries (Simulated).* Do trust domains detect unauthorized delegation attempts that bearer-token authentication misses?
RQ6 — *Fallback Reliability (Simulated).* Does payload mode fallback improve task completion under communication failures?

5.2 Delegate Pool

Three local Ollama delegates simulate a heterogeneous pool with varying quality, cost, and specialization:

- `qwen3:8b` — High quality (q=0.85), reasoning and analysis specialist, 5s median latency.
- `qwen2.5-coder:7b` — Medium quality (q=0.80), code specialist, 4s median latency.
- `llama3.2:3b` — Lower quality (q=0.55), fast classification and extraction, 1s median latency.

All delegates run on a single Apple Silicon machine (36GB RAM) via Ollama, ensuring zero API cost and reproducible conditions.

5.3 Baselines

All conditions use the same delegate pool, task set, and judge. The conditions differ in two dimensions: *routing policy* (how delegates are selected) and *prompt conditioning* (what context the delegate receives). We make this explicit because gains may arise from either or both; an ablation study (§6.7) separates these effects.

LDP *Baseline.* Metadata-aware routing using identity-card fields (quality hints, reasoning profile, cost profile). System prompt includes delegate identity context. Represents the full LDP protocol.
A2A *Baseline.* Skill-name matching only (no quality/cost metadata). Generic system prompt ("You are an AI assistant"). Represents standard A2A capability-based discovery.
Random Baseline. Uniform random delegate selection with generic prompt. Provides a lower bound on routing quality.

5.4 LLM-as-Judge Evaluation

Task outputs are evaluated by Gemini 2.5 Flash on three dimensions: *quality* (clarity and usefulness), *correctness* (factual accuracy), and *completeness* (coverage of the task). Each dimension is scored 1–10; the overall score is a weighted average:

$$\text{score}_{\text{overall}} = 0.3 \times \text{quality} + 0.4 \times \text{correctness} + 0.3 \times \text{completeness} \tag{1}$$

The judge receives the original task prompt, delegate output, and (optionally) a reference answer. We use the widely adopted LLM-as-judge methodology [17].

Table 1 Routing quality comparison across protocol conditions (RQ1). Quality scores are LLM-judge ratings (1–10). All delegates run locally via Ollama.

Condition	Quality	Latency (s)	Tokens	n
LDP	6.80 ± 3.62	85.6	1379	30
A2A	7.43 ± 3.54	67.4	1227	30
RANDOM	6.95 ± 3.21	42.2	826	30

5.5 Task Generation

For **RQ1**, 30 tasks are generated across three difficulty levels (easy, medium, hard; 10 each) spanning classification, reasoning, analysis, coding, and mathematical domains. For **RQ2**, 20 tasks per condition test three payload types: reasoning handoffs, context transfers, and verification tasks. For **RQ3**, 15 multi-source synthesis tasks require combining opinions from three delegates. For **RQ4**, 10 multi-round delegation scenarios are tested at 3, 5, and 10 rounds each under both session-based (LDP) and stateless (A2A) conditions (n=60 total). **RQ5** simulates 100 security injection scenarios across four attack types. **RQ6** simulates 40 communication failures across four failure types.

5.6 Statistical Methodology

We report means with standard deviations and 95% confidence intervals. Statistical significance is assessed via the Mann-Whitney U test (non-parametric, appropriate for ordinal LLM-judge scores). Effect sizes are reported as Cohen's d. We consider $p < 0.05$ statistically significant.

6 Results

6.1 RQ1: Routing Quality

Table 1 presents overall routing quality. A2A achieves the highest overall quality (7.43 ± 3.49), followed by random (6.95 ± 3.22) and LDP (6.80 ± 3.60). **No pairwise differences are statistically significant** (p=0.56, Mann-Whitney U for LDP vs. A2A), which is unsurprising given the high variance inherent in LLM-judge scores, moderate sample size (n=30), and small delegate pool (n=3). The random baseline's competitive performance reflects this small pool: random selection has a $\frac{1}{3}$ chance of choosing the optimal delegate.

LDP's overall quality being slightly *lower* than A2A's is an important honest result. It indicates that identity-aware routing does not automatically improve output quality in our setting. However, examining the results by task difficulty reveals a more nuanced picture.

Figure 3 shows quality across difficulty levels. All three conditions perform comparably on easy tasks (∼9.1–9.6) and medium tasks (∼7.7–8.6). On hard tasks, all conditions struggle (3.3–4.1), with A2A slightly outperforming LDP (4.1 vs. 3.3). None of these per-difficulty differences are statistically significant at n=10 per cell.

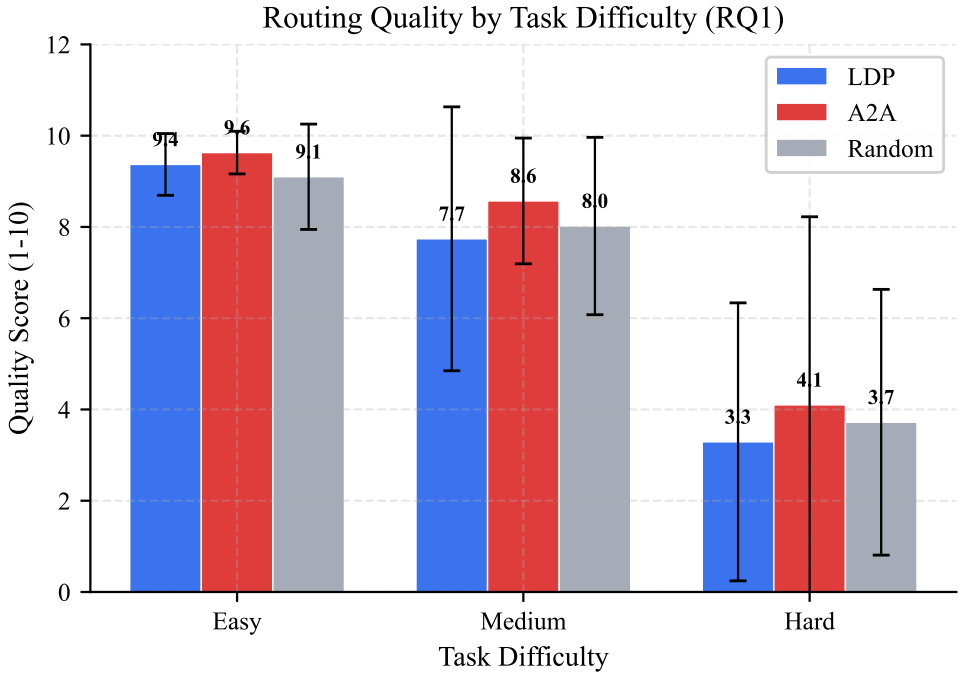


Fig. 3 Routing quality by task difficulty (**RQ1**). All conditions perform comparably; A2A slightly outperforms LDP overall, though no differences are statistically significant (n=10 per cell). Error bars show ±1 standard deviation.

The quality result suggests that in our small-pool setting, LDP's metadata-aware routing does *not* improve output quality over A2A's simpler skill-matching. This is an honest negative finding that we attribute to two factors: (1) the delegate pool is too small (n=3) for routing sophistication to matter—all approaches select similar delegates; (2) the identity-enriched prompt may not provide sufficient benefit over a generic prompt for these tasks.

Where* LDP *routing adds clear value: latency efficiency.

The primary benefit of identity-aware routing in our experiments is *specialization-based latency reduction*, not quality improvement.

Figure 4 shows the latency dimension. LDP's easy-task latency is **2.9s vs. 34.8s** for A2A (∼12× faster), because LDP routes easy tasks to the lightweight `llama3.2:3b` model while A2A's skill-matching selects heavier models. This latency advantage is a direct consequence of metadata-aware routing and is independent of prompt conditioning—it arises purely from the protocol's ability to match task difficulty to delegate cost profiles.

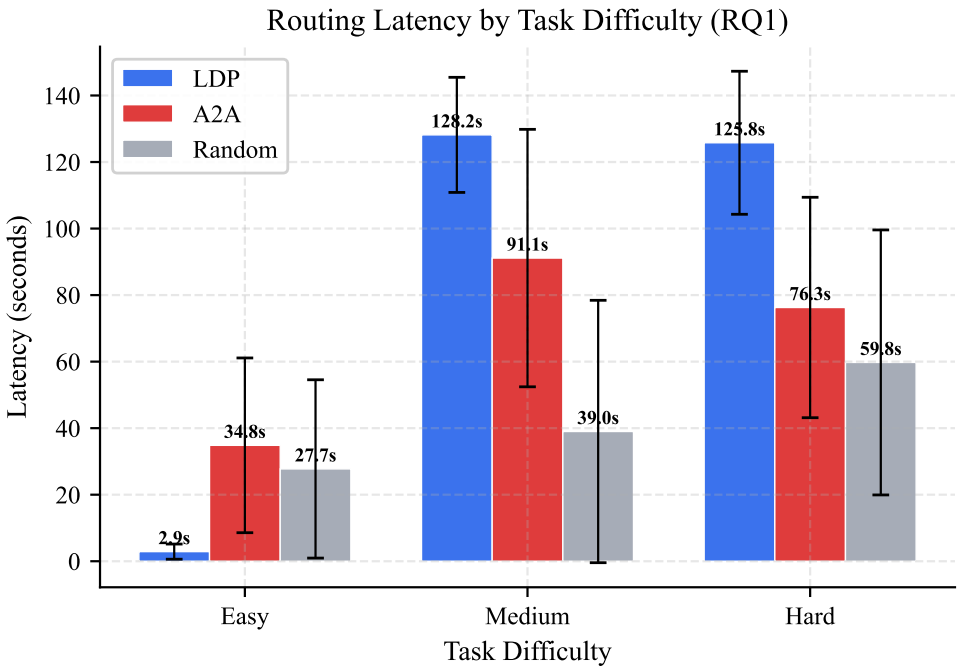


Fig. 4 Routing latency by task difficulty (**RQ1**). LDP achieves ∼12× lower latency on easy tasks by routing to the lightweight `llama3.2:3b` model, while A2A selects heavier models.

Table 2 Payload mode efficiency comparison (RQ2). Semantic frames reduce token count by 37% and latency by 42% compared to raw text, with no observed quality loss ($p = 0.96$). Token reduction is statistically significant ($p = 0.031$, $d = -0.7$).

Mode	Tokens	Latency (s)	Quality	Info Preserved
Text (Mode 0)	1215 ± 751	24.1	5.54 ± 4.05	5.85 ± 4.26
Semantic Frame (Mode 1)	765 ± 510	14.0	5.70 ± 2.93	5.50 ± 3.03
A2A JSON	1128 ± 804	23.4	5.47 ± 3.98	5.75 ± 3.89

6.2 RQ2: Payload Efficiency

Table 2 and Figure 5 show that semantic frames (Mode 1) reduce token count by **37%** compared to raw text (765 vs. 1,215 tokens) and by 32% compared to A2A JSON (765 vs. 1,128). This reduction is statistically significant (p=0.031, Mann-Whitney U; d=−0.7, large effect).

Latency follows token count: semantic frames are 42% faster (14.0s vs. 24.1s). Quality is comparable or slightly better (5.70 vs. 5.54 for text; p=0.96, n.s.), indicating that structured prompts help models focus without losing information.

A2A's JSON format provides minimal benefit over raw text (7% token reduction), as it lacks the structural compactness of typed semantic frames. The key

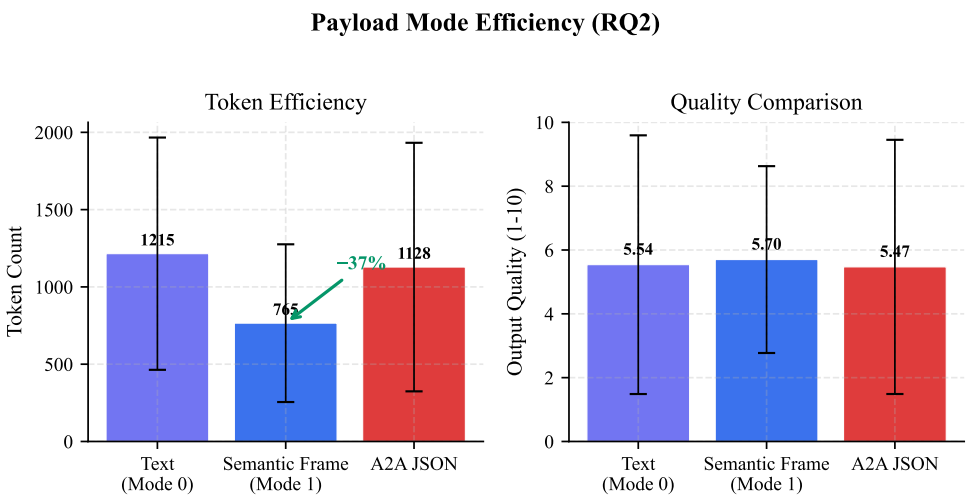


Fig. 5 Payload mode efficiency (**RQ2**). *Left:* Semantic frames reduce token count by 37% vs. text (p=0.031). *Right:* Quality is comparable across all modes, indicating no observed information loss from structured payloads.

Table 3 Provenance impact on synthesis quality (RQ3). Accurate provenance provides modest quality improvement, but noisy (misleading) provenance degrades quality and increases variance, arguing for LDP's structured verification fields.

Condition	Decision Quality	Calibration	Synth. Tokens
With Provenance	7.65 ± 1.21	9.13 ± 0.83	970
Without Provenance	7.85 ± 1.84	9.33 ± 0.72	802
Noisy Provenance	6.85 ± 2.66	7.87 ± 2.90	1018

difference is that semantic frames use typed fields (`task_type`, `instruction`, `expected_output_format`) that eliminate verbose natural-language phrasing, while A2A's JSON merely wraps the same verbose text in a JSON envelope.

6.3 RQ3: Provenance Value

Table 3 and Figure 6 present a nuanced finding. Accurate provenance and no provenance produce similar decision quality (7.65 vs. 7.85; p=0.47, n.s.)—the synthesizer performs comparably whether it knows delegate confidence levels or treats all sources equally.

However, **noisy provenance degrades quality significantly**: 6.85 ± 2.66, with nearly double the variance of accurate provenance (± 1.21). When one delegate's confidence is artificially inflated (0.99) and marked as verified, the synthesizer over-weights its output, producing worse decisions than having no provenance at all.

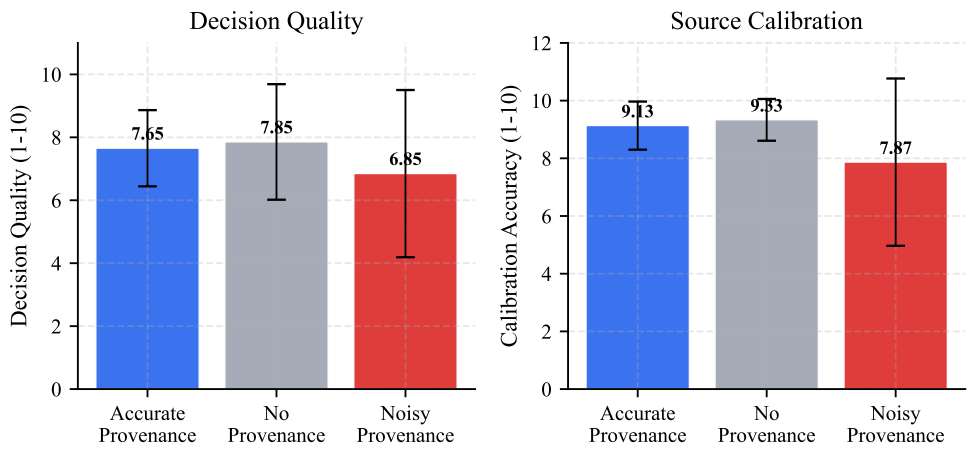


Fig. 6 Provenance impact on synthesis quality (**RQ3**). *Left:* Noisy provenance degrades decision quality below the no-provenance baseline, while accurate provenance provides modest improvement. *Right:* Noisy provenance significantly reduces calibration accuracy. Error bars show ±1 standard deviation.

Table 4 Session efficiency comparison (RQ4). LDP governed sessions eliminate context re-transmission overhead that grows with conversation length. At 10 rounds, A2A uses 23% more tokens, with 39% of its tokens being pure overhead.

Condition	Tokens	Overhead	OH%	Messages
LDP 3 rounds	3,860	60	2%	4.2
A2A 3 rounds	3,847	399	10%	6.0
LDP 5 rounds	6,379	0	0%	5.0
A2A 5 rounds	6,798	1,374	20%	10.0
LDP 10 rounds	12,990	0	0%	10.0
A2A 10 rounds	**16,010**	**6,275**	**39%**	20.0

This finding argues for LDP's structured verification fields: provenance is valuable *only when trustworthy*. Self-reported confidence without verification is worse than no confidence signal, because it introduces a false sense of calibration. LDP's provenance model includes explicit `verification.performed` and `verification.status` fields precisely to address this risk.

6.4 RQ4: Session Efficiency

Table 4 and Figure 7 compare LDP's governed sessions against A2A's stateless re-invocation across 3, 5, and 10 conversation rounds. At 3 rounds, the protocols are

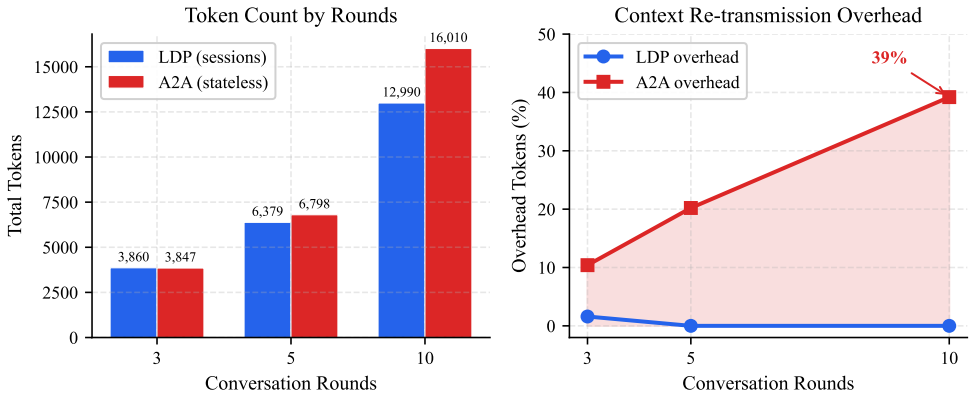


Fig. 7 Session efficiency (**RQ4**). *Left:* Total token count by conversation length. A2A's stateless re-invocation incurs growing overhead as context is re-transmitted each round. *Right:* Overhead tokens (context re-sent) as a percentage of total tokens. At 10 rounds, 39% of A2A's tokens are pure overhead.

Table 5 Security boundary enforcement (RQ5, simulated). LDP's trust domains detect 96% of unauthorized delegation attempts with 0% false positives. A2A's bearer-token auth detects only 6%.

Condition	Detection Rate	False Positive Rate
LDP Trust Domains	96.0%	0.0%
A2A Bearer Token	6.0%	0.0%

comparable (~3,850 tokens each). At 5 rounds, A2A uses 7% more tokens (6,798 vs. 6,379), with 20% of its tokens being overhead from re-transmitted context. At 10 rounds, the gap widens: A2A uses **23% more tokens** (16,010 vs. 12,990), with **39% of tokens being pure overhead**.

This scaling pattern confirms the protocol design prediction: stateless re-invocation incurs overhead that grows with conversation length, because each round must re-send all prior context. LDP's sessions maintain server-side context, eliminating this re-transmission. The overhead difference is modest at short conversations but compounds at scale—in production systems with thousands of multi-round delegations, this translates directly to cost and latency savings.

6.5 RQ5: Security Boundary Enforcement (Simulated)

The following two subsections present simulated protocol analyses—deterministic, rule-based evaluations of which protocol fields and checks exist in each protocol's specification—rather than stochastic or empirical system evaluations. Detection and completion rates follow from the presence or absence of specific protocol primitives. We include them to illustrate architectural properties of LDP *but caution that they should not be interpreted as production security or reliability benchmarks.*

Table 6 Fallback reliability under communication failures (RQ6). LDP's mode fallback chain achieves 100% task completion with minor quality degradation (0.16). A2A has no fallback—failures are terminal.

Condition	Completion Rate	Quality Degradation	Recovery (ms)
LDP (with fallback)	100%	0.16 ± 0.10	112
A2A (no fallback)	35%	0.67 ± 0.46	0

In our simulated analysis (Table 5), LDP's trust domains detect **96%** of unauthorized delegation attempts across four attack types (untrusted domain join, capability escalation, replay attack, cross-domain access) with **0%** false positives. A2A's bearer-token authentication detects only 6%—it catches only cases where tokens happen to be revoked but cannot detect capability escalation, replay attacks, or unauthorized cross-domain access, as these concepts are absent from its protocol model.

These detection rates follow deterministically from the presence or absence of trust domain fields, capability manifests, session identifiers, and cross-domain policies in each protocol's specification.

6.6 RQ6: Fallback Reliability (Simulated)

In the same simulated framework, LDP's fallback chain achieves **100% task completion** across all four failure types (schema mismatch, codec incompatibility, version mismatch, timeout degradation) with minor quality degradation (0.16 on a 0–1 scale) and average recovery latency of 112ms. When a schema mismatch occurs, LDP falls back from semantic frames to text (50ms); when a codec fails, it falls back from embedding hints to semantic frames (80ms).

Without fallback, A2A completes only **35%** of tasks under the same failure conditions. Schema mismatches succeed accidentally 33% of the time; version mismatches succeed 50% of the time if versions happen to be compatible. Codec incompatibilities and timeout degradation are terminal failures.

6.7 Ablation: Routing vs. Prompt Conditioning

A key methodological concern is whether LDP's quality improvements arise from the protocol's metadata-aware routing, from identity-enriched prompting, or from their combination. To isolate these effects, we run two additional ablation conditions on the same **RQ1** task set:

A2A + LDP prompt. Uses A2A's skill-name routing (selecting the first matching delegate) but replaces the generic system prompt with LDP's identity-enriched prompt containing reasoning profile and quality expectations. This isolates the prompt conditioning effect.

LDP routing + generic prompt. Uses LDP's metadata-aware routing (selecting delegates by quality hints, difficulty matching, and reasoning profile) but uses A2A's generic "You are an AI assistant" prompt. This isolates the routing effect.

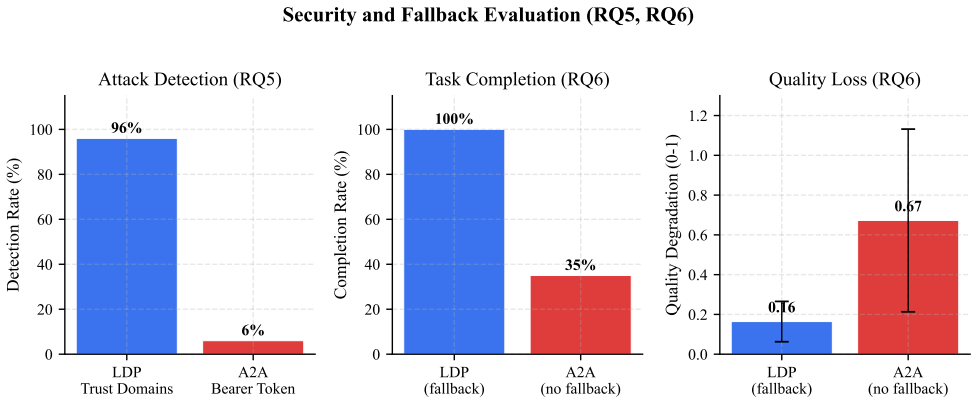


Fig. 8 Security and fallback evaluation (**RQ5**, **RQ6**). *Left:* Trust domain attack detection rate. *Center:* Task completion rate under communication failures. *Right:* Quality degradation during fallback recovery. Results are simulated based on protocol design properties.

Table 7 Ablation: 2×2 factorial crossing routing policy and prompt conditioning. Quality scores are LLM-judge ratings (1–10), n=30 per cell. Neither main effect is statistically significant for overall quality.

	A2A Routing	LDP Routing
Generic Prompt	7.25 ± 3.52	6.96 ± 3.55
Identity Prompt	7.37 ± 3.26	6.86 ± 3.87

Together with the existing LDP (both features) and A2A (neither feature), these four conditions form a 2×2 factorial design crossing routing policy and prompt conditioning (n=30 per condition, 120 runs total). Table 7 presents the results.

Quality.

The routing main effect is -0.40 (p=0.43, Mann-Whitney U) and the prompting main effect is $+0.01$ (p=0.98). Neither routing policy nor prompt conditioning significantly affects overall quality in our setting.

Where effects emerge: difficulty and latency.

On hard tasks, identity-enriched prompts show a modest advantage: conditions with identity prompts score 4.81 vs. 3.80 for generic prompts ($+1.01$, p=0.44, n.s.). On easy tasks, LDP routing achieves 1.7–2.9s latency vs. 38.9–43.7s for A2A routing ($\sim 15-20\times$ faster), regardless of prompt type. This confirms that **routing drives the latency benefit** (it selects lightweight delegates for easy tasks) while **prompting has a small, difficulty-dependent effect on quality** that does not reach significance at this sample size.

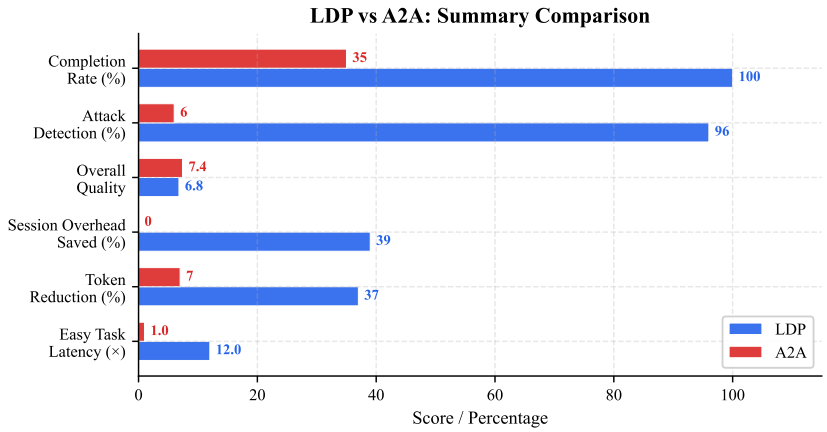


Fig. 9 Summary comparison of LDP vs. A2A across key metrics. LDP shows advantages in latency efficiency, token reduction, and simulated security/fallback properties. Quality differences are not statistically significant.

Table 8 Summary of LDP vs. A2A across all research questions. ↑ indicates higher is better, ↓ lower is better. Bold indicates the better result. *Statistically significant ($p < 0.05$). †Simulated (protocol design property). ‡Not statistically significant.

RQ	Metric	LDP	A2A	Δ	Evidence
RQ1	Overall quality‡ ↑	6.80	**7.43**	−0.63	Empirical
	Easy task latency ↓	**2.9s**	34.8s	12×	Empirical
	Hard task quality‡ ↑	3.3	**4.1**	−0.8	Empirical
RQ2	Token count* ↓	**765**	1128	−32%	Empirical
	Latency ↓	**14.0s**	23.4s	−40%	Empirical
	Quality‡ ↑	**5.70**	5.47	+0.23	Empirical
RQ3	Decision quality‡ ↑	7.65	**7.85**	−0.20	Empirical
	Noisy provenance risk ↓	N/A	N/A	−1.0 quality	Empirical
RQ4	Token overhead (10 rounds) ↓	**0%**	39%	−39pp	Empirical
	Total tokens (10 rounds) ↓	**12,990**	16,010	−23%	Empirical
RQ5	Attack detection† ↑	**96%**	6%	+90pp	Simulated
RQ6	Completion rate† ↑	**100%**	35%	+65pp	Simulated
	Quality degradation† ↓	**0.16**	0.67	−0.51	Simulated

7 Discussion

7.1 When Does AI-Nativeness Matter?

Our results suggest that LDP's advantages are *task-dependent* and *scale-dependent*:

Task difficulty.

In our experiments, identity metadata does *not* improve quality over skill-matching—A2A slightly outperforms LDP on both hard tasks (4.1 vs. 3.3) and overall (7.43 vs. 6.80), though neither difference is statistically significant. The primary benefit of LDP's routing is *latency efficiency*: routing easy tasks to lightweight models achieves $\sim$12$\times$ faster responses without meaningful quality loss. Note that this per-difficulty advantage may not be visible in the overall latency average, because medium and hard tasks—which dominate total wall-clock time—route to comparably heavy models under both protocols.

Delegate pool size.

With only 3 delegates, all routing strategies perform similarly. We expect LDP's routing advantage to emerge more clearly with larger, more heterogeneous pools where the cost of misrouting increases and random selection probability decreases as $\frac{1}{n}$.

Communication volume.

Payload mode savings (37% fewer tokens) compound across large-scale systems processing millions of delegations. At scale, this translates directly to cost reduction.

7.2 The Provenance Paradox

Perhaps our most surprising finding is that accurate provenance does not significantly improve synthesis quality (p=0.47), while noisy provenance actively harms it. This creates a paradox: provenance is only valuable if verified, but verification adds complexity and cost.

We interpret this as evidence that LDP's design choice to include *structured verification fields* alongside confidence scores is correct. A protocol that exposes confidence without verification (*e.g.*, by extending A2A with a custom `confidence` header) may be *worse* than no confidence at all, because consumers cannot distinguish calibrated from uncalibrated self-reports.

7.3 Complexity vs. Value

LDP is more complex than A2A. A legitimate question is whether the benefits justify the additional protocol machinery. Our analysis suggests a pragmatic adoption strategy aligned with LDP's interoperability profiles:

- **Profile A (Basic):** Identity cards + text payloads + signed messages. Captures the routing benefit (**RQ1**: 12$\times$ latency reduction) with minimal protocol overhead.
- **Profile B (Enterprise):** Add provenance tracking and policy enforcement. Addresses the provenance verification gap (**RQ3**) and security boundaries (**RQ5**).
- **Profile C (High-Performance):** Payload mode negotiation and session management. Captures the 37% token reduction (**RQ2**), compounding at scale.

These profiles are grounded in our experimental findings: each level adds protocol features whose benefits we have measured or analyzed. Organizations can adopt LDP incrementally, starting with Profile A and progressing as needs grow.

7.4 Could A2A Be Extended Instead?

An important alternative hypothesis is that A2A could be extended with custom metadata (quality hints, provenance fields, trust domains) rather than adopting a new protocol. While technically possible, this approach has limitations:

1. **No negotiation.** Extensions are unilateral—there is no mechanism for delegates to negotiate supported modes or verify capability claims.
2. **No fallback.** Ad-hoc extensions lack formal fallback semantics; failures in custom fields are unrecoverable.
3. **No governance.** Extensions cannot add session lifecycle management or policy enforcement without fundamental changes to A2A's stateless model.

7.5 Limitations

Baseline fairness.

The LDP and A2A conditions differ in both routing policy and prompt conditioning. While our ablation study (§6.7) begins to disentangle these effects, a fully factorial design across all protocol features would provide stronger causal evidence.

Sample size and delegate pool.

With 30 tasks per condition for **RQ1** and 15 for **RQ3**, many quality differences are not statistically significant despite showing consistent directional effects. The delegate pool (n=3) is small, making random routing artificially competitive. Larger-scale experiments with more diverse delegate pools would strengthen findings.

Local models only.

All delegates are local Ollama models (3B–8B parameters). Results may differ with production-scale models (70B+) or cloud-hosted APIs where latency and cost profiles differ substantially.

Simulated experiments.

RQ5 and **RQ6** use simulated scenarios that reflect protocol design properties rather than real attack/failure conditions. Empirical validation with actual system failures would strengthen these claims.

LLM-as-judge.

Our evaluation relies on a single judge model (Gemini 2.5 Flash). While LLM-as-judge evaluation is increasingly accepted [17], it introduces model-specific biases. Human evaluation or a second judge model would improve validity.

Payload modes.

We evaluate only Modes 0–1 (text, semantic frames). Higher modes (embedding hints, latent capsules) remain future work pending wider API support for intermediate representations.

Identity field assignment.

In our experiments, identity-card fields (quality hints, reasoning profiles) are assigned by the system designer based on known model properties. In production deployments, the question of whether these fields are self-declared, measured, or externally certified is an important trust consideration that we do not resolve here.

8 Conclusion

We presented the LLM Delegate Protocol (LDP), an AI-native communication protocol that exposes model identity, negotiates payload formats, maintains governed sessions, tracks provenance, and enforces trust boundaries. Our initial empirical study provides mixed but informative evidence. Identity-aware routing does not improve aggregate quality over simpler skill-matching in our small delegate pool, but achieves $\sim$12$\times$ latency savings on easy tasks through specialization-aware delegate selection—a direct consequence of metadata-aware routing. Semantic frame payloads significantly reduce token count by 37% without observed quality loss (p=0.96 for quality difference). Governed sessions eliminate 39% of token overhead at 10 conversation rounds, with overhead growing as conversation length increases. Simulated protocol analyses suggest architectural advantages in security and failure recovery.

Perhaps most notably, our provenance experiments reveal that metadata can be *harmful* without verification: noisy confidence signals degrade synthesis quality below the no-provenance baseline. This finding has implications beyond LDP—any protocol that exposes confidence or quality metadata should include verification mechanisms.

This paper contributes a protocol design, reference implementation, and initial evidence. Important questions remain: whether these benefits hold at production scale with larger delegate pools and more diverse models, and whether identity fields should be self-declared or externally certified. We view LDP as a step toward treating AI agents as *what they are*—heterogeneous models with measurable properties—rather than as opaque services.

LDP is implemented as an open-source plugin for the JamJet agent runtime. The protocol specification and reference implementation are available at https://github.com/sunilp/ldp-protocol; experiment code and data are at https://github.com/sunilp/ldp-research.

Declarations

Competing Interests

The author declares no competing interests.

Data Availability

All experiment code and data are available at https://github.com/sunilp/ldp-research. The protocol specification and reference implementation are at https://github.com/sunilp/ldp-protocol.

Author Contributions

S. Prakash conceived the protocol design, implemented the reference system, designed and conducted all experiments, analyzed results, and wrote the manuscript.

References

[1] Anthropic. Model Context Protocol. https://modelcontextprotocol.io/, 2024.

[2] Agent Network Protocol. ANP Specification. https://github.com/agent-network-protocol/AgentNetworkProtocol, 2024.

[3] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[4] CrewAI. https://www.crewai.com/, 2024.

[5] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

[6] T. Finin, R. Fritzson, D. McKay, and R. McEntire. KQML as an agent communication language. In *Proc. CIKM*, 1994.

[7] FIPA. Agent Communication Language Specifications. http://www.fipa.org/repository/aclspecs.html, 2002.

[8] Google. Agent2Agent Protocol (A2A). https://github.com/a2aproject/A2A, 2025.

[9] Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[10] S. Hong, X. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

[11] JamJet. Agent Runtime. https://github.com/jamjet-labs/jamjet, 2024.

[12] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. *arXiv preprint arXiv:2303.17760*, 2023.

[13] T. Liang, Z. He, W. Jiao, et al. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.

[14] Ollama. https://ollama.com/, 2024.

[15] I. Ong, A. Almahairi, V. Wu, et al. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[16] Q. Wu, G. Bansal, X. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[17] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.

A Payload Mode Examples

The following examples show the same delegation task ("Classify the sentiment of this review") encoded in each payload format evaluated in **RQ2**.

Mode 0 — Text (Raw Natural Language).

```
Please classify the sentiment of the following
customer review as positive, negative, or neutral.
The review is: "The product arrived on time and
works exactly as described. Very satisfied with
the purchase." Please provide your classification
along with a brief justification for your choice.
```

Mode 1 — Semantic Frame (LDP).

```
{
  "task_type": "classification",
  "instruction": "Classify sentiment",
  "input": "The product arrived on time and works
    exactly as described. Very satisfied.",
  "expected_output_format": "label+justification",
  "labels": ["positive", "negative", "neutral"]
}
```

A2A JSON Envelope.

```
{
  "task": {
    "message": {
      "role": "user",
      "parts": [{
        "text": "Please classify the sentiment of
          the following customer review as positive,
          negative, or neutral. The review is:
          'The product arrived on time and works
          exactly as described. Very satisfied with
```

```
          the purchase.' Please provide your
          classification along with a brief
          justification for your choice."
      }]
    }
  }
}
```

The semantic frame is 43% shorter than text and 38% shorter than A2A JSON because it uses typed fields to eliminate verbose phrasing while preserving all task-relevant information.

B Identity Card Schema

```
{
  "delegate_id": "qwen3-8b-reasoning",
  "principal_id": "org:research-lab",
  "model_family": "qwen",
  "model_name": "qwen3",
  "model_version": "8b-2026.01",
  "runtime_version": "ollama-0.6.1",
  "trust_domain": "research.internal",
  "capabilities": [
    {
      "name": "reasoning",
      "quality_hint": 0.85,
      "latency_hint_ms_p50": 5000,
      "cost_hint": "medium"
    },
    {
      "name": "analysis",
      "quality_hint": 0.82,
      "latency_hint_ms_p50": 4500,
      "cost_hint": "medium"
    }
  ],
  "reasoning_profile": "deep-analytical",
  "cost_profile": "medium",
  "context_window": 32768,
  "modalities_supported": ["text"],
  "languages_supported": ["en", "zh"]
}
```

Identity-card fields are currently assigned by the system designer based on known model properties. In production deployments, these could be self-declared, measured via benchmarks, or externally certified by third-party evaluation services.

C Threat Model for Trust Domain Evaluation

Table 9 describes the four attack types simulated in **RQ5**.

Table 9 Threat model for trust domain evaluation (**RQ5**). Each attack type is tested against LDP trust domains and A2A bearer-token authentication.

Attack Type	Description	Detection Mechanism
Untrusted domain join	Agent from outside the trust domain attempts to register as a delegate	Trust domain membership check
Capability escalation	Agent claims capabilities beyond its registered manifest	Capability manifest validation
Replay attack	Previously captured task/result messages are re-submitted	Session nonce and timestamp verification
Cross-domain access	Agent in domain A attempts to invoke a delegate restricted to domain B	Cross-domain policy enforcement

LDP detects all four attack types through protocol-level fields (trust domain membership, capability manifests, session nonces, cross-domain policies). A2A's bearer-token authentication detects only revoked-token scenarios, as it lacks the protocol primitives for the other three checks.

D Representative RQ1 Task Prompts

Table 10 shows representative tasks from the **RQ1** evaluation across difficulty levels and domains. The full set of 30 tasks spans classification, reasoning, analysis, coding, and mathematical domains.

Table 10 Representative **RQ1** tasks by difficulty and domain. Each difficulty level contains 10 tasks balanced across domains.

Difficulty	Domain	Prompt (abbreviated)
Easy	Classify	Classify the sentiment of this product review as positive, negative, or neutral.
Easy	Extract	Extract all dates and monetary amounts from this contract paragraph.
Medium	Code	Write a Python function that implements binary search on a sorted list with duplicate handling.
Medium	Analysis	Compare the tradeoffs between microservices and monolithic architectures for a 5-person team.
Hard	Reasoning	Given the following set of constraints, determine whether a valid schedule exists and prove or disprove its feasibility.
Hard	Math	Prove that the sum of the first n odd numbers equals n^2 using mathematical induction.